

API Testing Report

Week 5 — API Testing with Postman | JSONPlaceholder REST API

1. Project Overview

This report documents a self-directed API testing project completed as part of a structured 7-day QA learning plan (Week 5). The objective was to build practical, hands-on skills in API testing using Postman, targeting the public JSONPlaceholder REST API (<https://jsonplaceholder.typicode.com>) as a safe, free practice environment.

The project covered the full request lifecycle — GET, POST, PUT, PATCH, and DELETE methods — along with status code validation, response body validation, query parameters, request headers, and both positive and negative testing techniques.

2. Scope

- API Under Test: JSONPlaceholder (<https://jsonplaceholder.typicode.com>)
- Resources tested: /users, /posts
- HTTP methods covered: GET, POST, PUT, PATCH, DELETE
- Testing tool: Postman (collections, environments, Params, Scripts/Tests)
- Test types: Positive testing, Negative testing, Status code validation, Response body validation, Header validation, Query parameter validation
- Total test cases executed: 20

3. Methodology

Testing was performed manually in Postman, organized into a collection named **"Week 5 - API Testing"**, structured into three folders:

- **Users** — happy-path (positive) requests: retrieving, creating, updating, and deleting users, plus query-parameter search.
- **Negative Testing** — invalid endpoints, non-existent resource IDs, and malformed request bodies, to verify how the API behaves under incorrect or unexpected conditions.
- **Posts** — testing a second resource to confirm consistent API behavior beyond a single endpoint.

Each request was validated using Postman test scripts (pm.test) asserting status codes and, where applicable, specific response fields, header values, or response structure (e.g., array vs. object, empty vs. populated).

4. Test Summary

Metric	Count
Total Test Cases	20
Passed	17
Failed	3
Pass Rate	85%

5. Test Case Details

The complete set of 20 executed test cases is summarized below. Full step-by-step test scripts are maintained in the accompanying Postman collection and Week_5_API_Test_Cases.xlsx file.

ID	Method	Endpoint	Scenario	Exp.	Act.	Result
TC_API_001	GET	/users	Verify users can be retrieved	200	200	PASS
TC_API_002	GET	/users/1	Verify a user can be retrieved using a valid ID	200	200	PASS
TC_API_003	GET	/users/5	Verify a user can be retrieved using another valid ID	200	200	PASS
TC_API_004	GET	/users/999	Verify behavior when requesting a non-existing user	404	404	PASS
TC_API_005	POST	/users	Verify that a new user can be created	201	201	PASS
TC_API_006	POST	/users	Verify behavior when creating a user with missing email	201	201	PASS
TC_API_007	PUT	/users/1	Verify an existing user's details can be updated	200	200	PASS
TC_API_008	DELETE	/users/1	Verify that a user can be deleted	200	200	PASS
TC_API_009	GET	/users/1	Verify response contains required fields (id, name)	200	200	PASS
TC_API_010	GET	/users	Verify users can be filtered using username query param	200	200	PASS
TC_API_011	POST	/users	Verify API returns JSON content type after creating a user	201	201	PASS
TC_API_012	GET	/invalidendpoint	Verify behavior when requesting a non-existent endpoint	404	404	PASS
TC_API_013	PUT	/users/9999	Verify behavior when updating a non-existent user	404	500	FAIL
TC_API_014	DELETE	/users/9999	Verify behavior when deleting a non-existent user	200	200	PASS
TC_API_015	POST	/users	Verify behavior when request body has malformed JSON	400	500	FAIL
TC_API_016	PATCH	/users/1	Verify a user's email can be partially updated	200	200	PASS
TC_API_017	PATCH	/users/9999	Verify behavior when patching a non-existent user	404	200	FAIL
TC_API_018	POST	/users	Verify behavior when creating a user with extra fields	201	201	PASS
TC_API_019	GET	/users	Verify behavior when query parameter matches no users	200	200	PASS
TC_API_020	GET	/posts	Verify that all posts can be retrieved successfully	200	200	PASS

6. Bugs Found

During negative testing, three instances were identified where JSONPlaceholder's actual behavior did not match the expected behavior of a well-designed REST API. These are documented below as defects, in the same spirit as production QA bug reporting — even though JSONPlaceholder is a public mock API and these findings will not be formally fixed.

BUG_API_001: PUT on a non-existent user returns 500 instead of 404

Related Test Case	TC_API_013
Steps to Reproduce	Send PUT /users/9999 with a valid JSON body for a user ID that does not exist.
Expected Result	API should return 404 Not Found, since the target resource does not exist.
Actual Result	API returns 500 Internal Server Error.
Severity	Medium
Impact / Notes	A 500 error suggests unhandled server-side failure rather than a controlled validation response, which could mask real server issues and gives API consumers no clear indication the resource simply doesn't exist.

BUG_API_002: Malformed JSON in POST request body returns 500 instead of 400

Related Test Case	TC_API_015
Steps to Reproduce	Send POST /users with an intentionally malformed (syntactically invalid) JSON body.
Expected Result	API should return 400 Bad Request, indicating a client-side input error.
Actual Result	API returns 500 Internal Server Error.
Severity	Medium
Impact / Notes	Client input errors should be classified as 4xx, not 5xx. A 500 response implies a server fault rather than a bad request, which can mislead monitoring/alerting systems in production APIs.

BUG_API_003: PATCH on a non-existent user returns 200 instead of 404

Related Test Case	TC_API_017
Steps to Reproduce	Send PATCH /users/9999 with a partial update body for a user ID that does not exist.
Expected Result	API should return 404 Not Found.
Actual Result	API returns 200 OK.
Severity	Low
Impact / Notes	Inconsistent existence-checking depending on HTTP method — PUT fails hard (500) while PATCH silently succeeds (200) for the same non-existent resource.

Observation: All three defects share a common root cause — JSONPlaceholder does not consistently validate whether a resource exists before processing PUT and PATCH requests, and does not validate request body syntax before processing POST requests. In a real production API, this pattern would be flagged as a priority fix, since consistent, correct status codes are essential for API consumers to handle errors reliably.

7. Conclusion

This project provided hands-on practice across the full range of core API testing skills: constructing and sending requests for all major HTTP methods, validating status codes and response bodies, testing query parameters and headers, and — most importantly — applying a negative-testing mindset to uncover real inconsistencies in API behavior. Of 20 test cases executed, 17 passed and 3 uncovered genuine defects, demonstrating that thorough negative testing adds real value beyond simple happy-path checks.

Supporting artifacts for this report: `Week_5_API_Test_Cases.xlsx` (full test case log), `Week_5_API_Testing_Collection.json` (exported Postman collection), and portfolio screenshots (01–05) demonstrating the collection structure, request/response examples, negative testing, and automated test assertions.